

Cloud Native Oslo 2026

Gitless GitOps

Stefan Prodan

Flux maintainer

Francesco Beltramini

Flux connoisseur



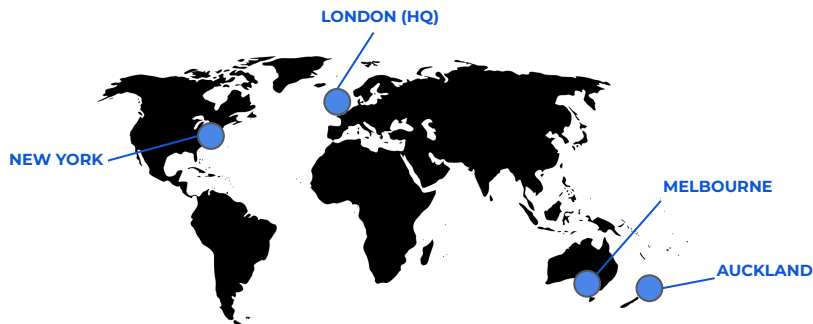


Head of Development
ControlPlane

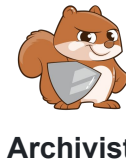


Field CTO @
ControlPlane

ControlPlane is a *highly specialised*
Enterprise DevSecOps and AI Security Consultancy
founded to **design, build, and secure Cloud Native solutions**
for *regulated* organisations



Enterprise
for
Open Source



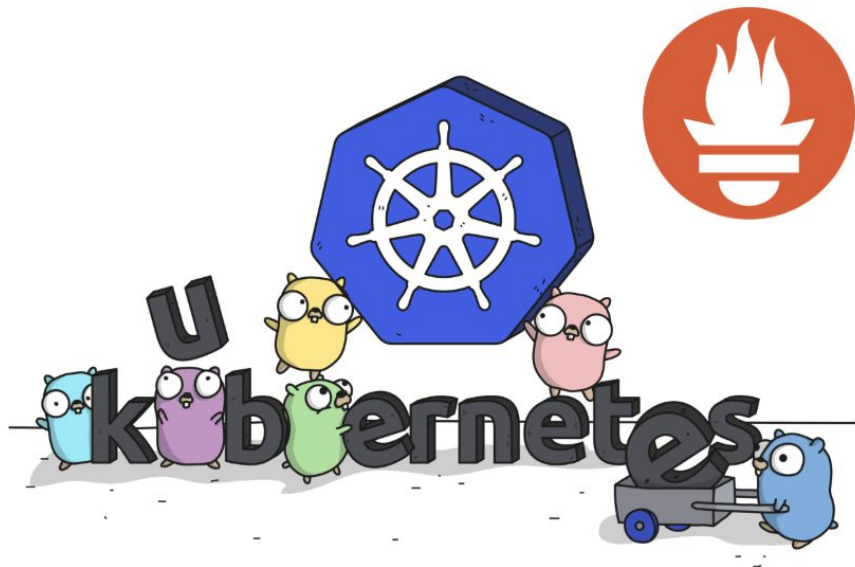
GitOps Inception - Weaveworks 2016

Declarative vs Imperative

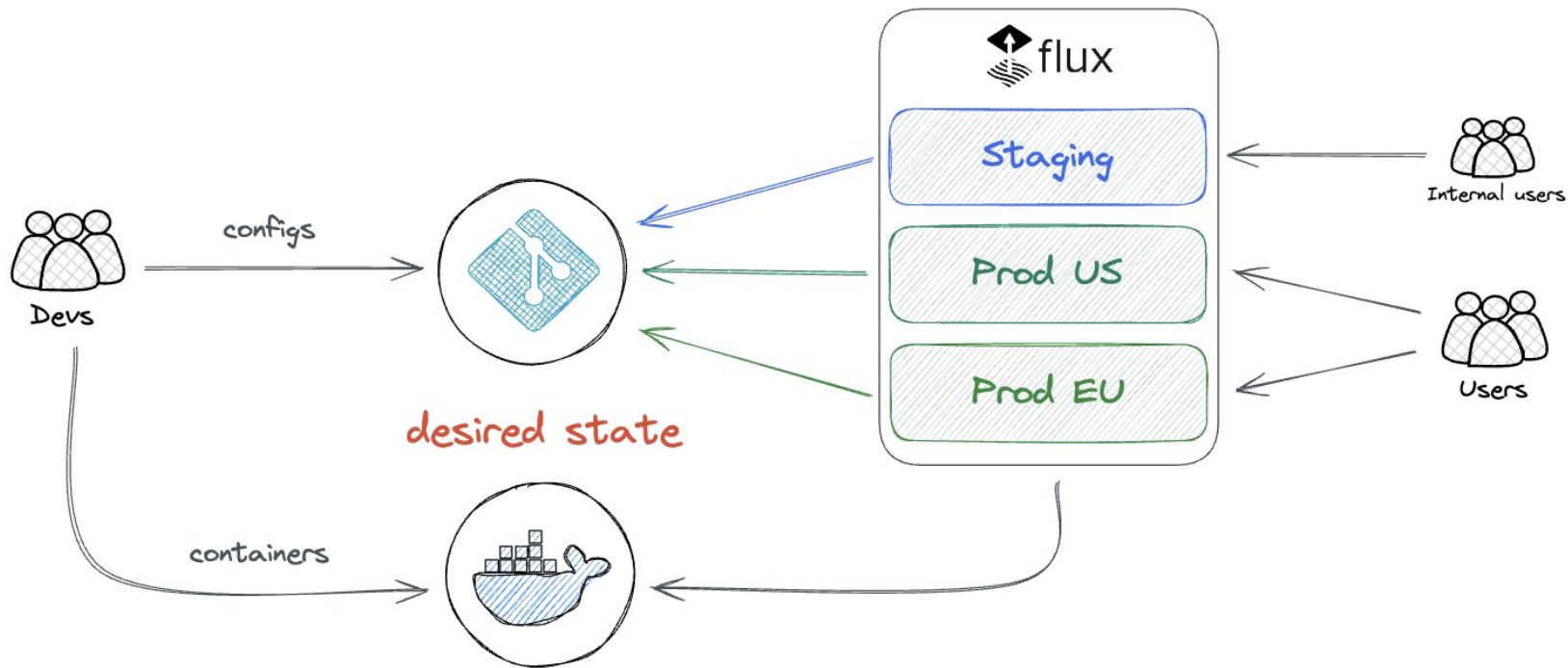
Pull vs Push

Everyone knows Git so
we'll replace kubectl with git

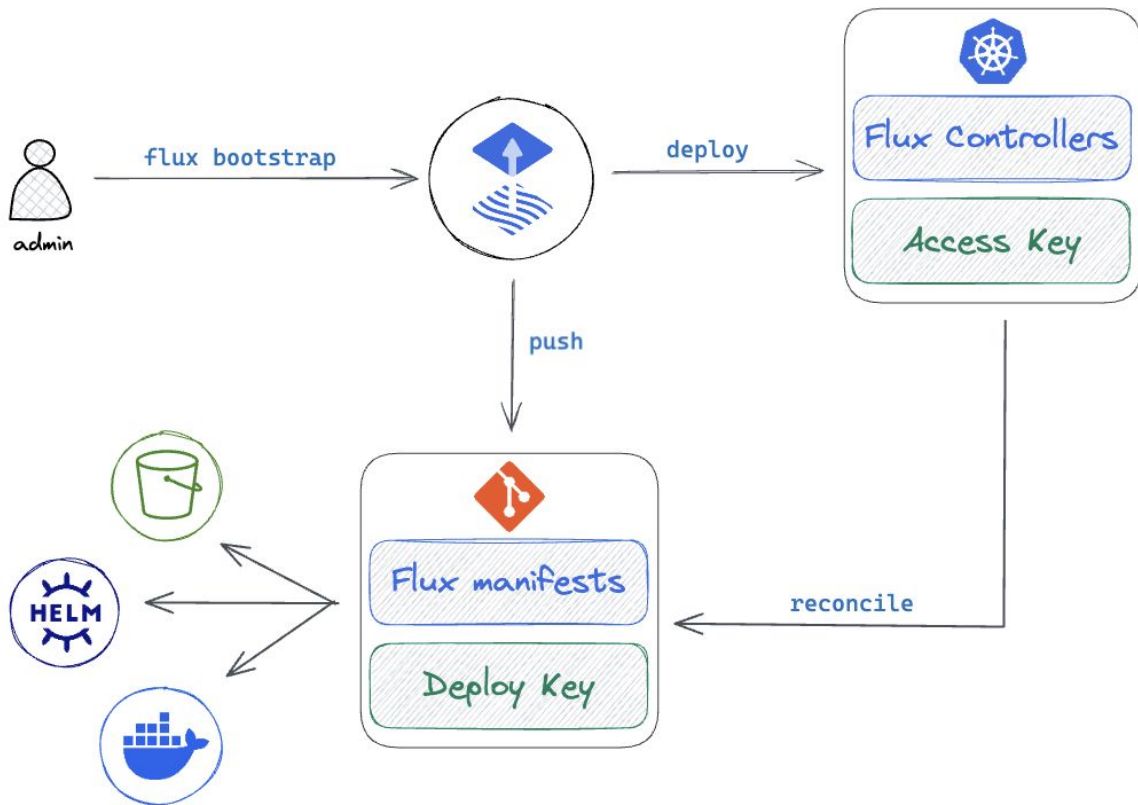
Devs will love **operations** once we teach
them Kubernetes and Prometheus



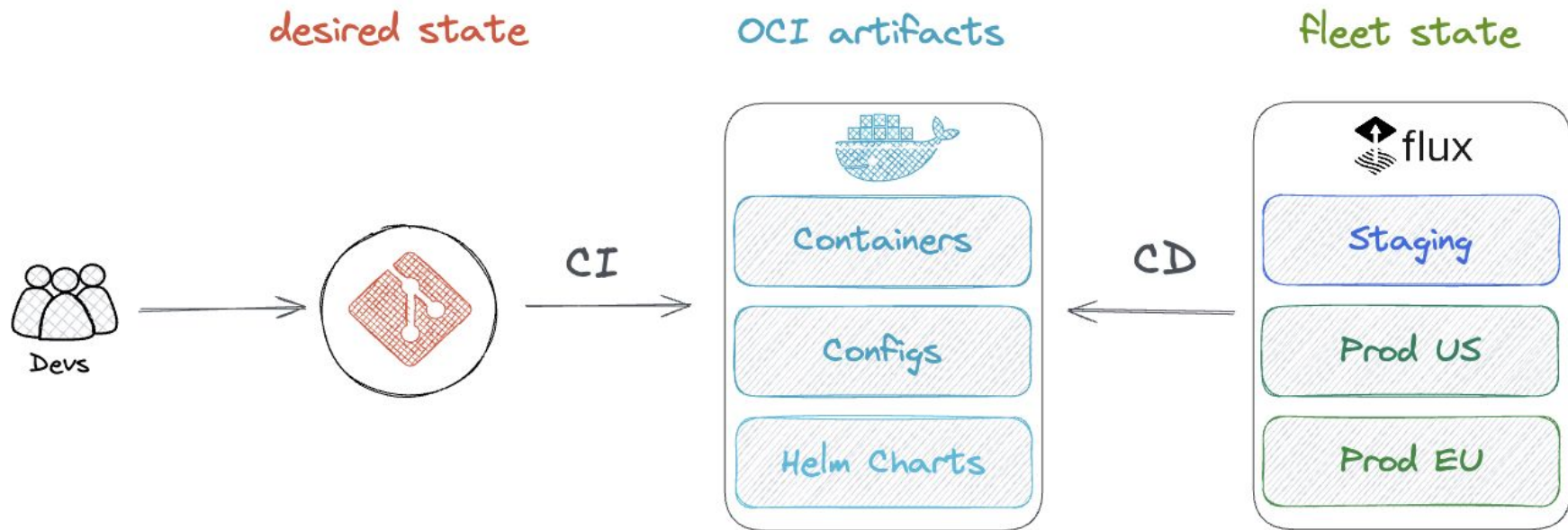
Flux - GitOps Pipeline



Flux - GitOps Bootstrap



Flux - Gitless GitOps Pipeline



Flux - GitRepository → OCIRepository

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: GitRepository
metadata:
  name: podinfo
  namespace: default
spec:
  interval: 5m
  url: https://github.com/stefanprodan/podinfo
  ref:
    semver: '>=1.0.0'
  verify:
    mode: HEAD
    secretRef:
      name: pgp-public-keys
```

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: OCIRepository
metadata:
  name: podinfo
  namespace: default
spec:
  interval: 5m
  url: oci://ghcr.io/stefanprodan/manifests/podinfo
  ref:
    semver: '>=1.0.0'
  verify:
    provider: cosign
    matchOIDCIdentity:
      - issuer: "^https://token.actions.githubusercontent.com$"
        subject: "^https://github.com/stefanprodan/podinfo.*$"
```

Flux - HelmChart → OCIRepository

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: HelmRepository
metadata:
  name: podinfo
  namespace: default
spec:
  interval: 5m
  url: https://stefanprodan.github.io/podinfo
---
apiVersion: source.toolkit.fluxcd.io/v1
kind: HelmChart
metadata:
  name: podinfo
  namespace: default
spec:
  interval: 5m
  chart: podinfo
  sourceRef:
    kind: HelmRepository
    name: podinfo
  version: '>=1.0.0'
```

```
apiVersion: source.toolkit.fluxcd.io/v1
kind: OCIRepository
metadata:
  name: podinfo
  namespace: default
spec:
  interval: 5m
  url: oci://ghcr.io/stefanprodan/charts/podinfo
  ref:
    semver: '>=1.0.0'
  layerSelector:
    mediaType: "application/vnd.cncf.helm.chart.content.v1.tar+gzip"
    operation: copy
  verify:
    provider: cosign
    matchOIDCIdentity:
      - issuer: "^https://token.actions.githubusercontent.com$"
        subject: "^https://github.com/stefanprodan/podinfo.*$"
```

Flux - Publishing OCI Artifacts

CI workflow

```
git clone https://github.com/org/app.git
cd app && git checkout v1.0.0

flux push artifact oci://ghcr.io/org/manifests/app:${git tag --points-at HEAD} \
  --path="./deploy" \
  --source="$(git config --get remote.origin.url)" \
  --revision="$(git tag --points-at HEAD)@sha1:${git rev-parse HEAD}"

cosign sign ghcr.io/org/manifests/app:${git tag --points-at HEAD}

flux tag artifact oci://ghcr.io/org/manifests/app:${git tag --points-at HEAD} \
  --tag=latest
```

Flux - Publishing with GitHub Actions

GitHub Workflow

```
- name: Push artifact
  uses: controlplaneio-fluxcd/distribution/actions/push@main
  id: push
  with:
    repository: ghcr.io/${{ github.repository }}/manifests
    path: "./deploy"
    diff-tag: "latest"
    tags: ${{ github.ref_name }}
- name: Sign artifact
  if: steps.push.outputs.pushed == 'true'
  run: cosign sign --yes $DIGEST_URL
  env:
    DIGEST_URL: ${{ steps.push.outputs.digest-url }}
```

Flux - Artifact Generators

```
apiVersion: source.extensions.fluxcd.io/v1beta1
kind: ArtifactGenerator
metadata:
  name: chart-composition
  namespace: apps
spec:
  sources:
    - alias: chart
      kind: OCIRepository
      name: podinfo-chart
    - alias: repo
      kind: OCIRepository
      name: podinfo-values
  artifacts:
    - name: podinfo-composite
      originRevision: "@chart"
      copy:
        - from: "@chart/"
          to: "@artifact/"
        - from: "@repo/charts/podinfo/values.yaml"
          to: "@artifact/podinfo/values.yaml"
          strategy: Overwrite
        - from: "@repo/charts/podinfo/values-prod.yaml"
          to: "@artifact/podinfo/values.yaml"
          strategy: Merge
```

```
apiVersion: source.extensions.fluxcd.io/v1beta1
kind: ArtifactGenerator
metadata:
  name: monorepo-decomposition
  namespace: platform
spec:
  sources:
    - alias: monorepo
      kind: OCIRepository
      name: platform-monorepo
  artifacts:
    - name: policies
      copy:
        - from: "@monorepo/platform/policy/**"
          to: "@artifact/"
    - name: auth-service
      copy:
        - from: "@monorepo/services/auth/deploy/**"
          to: "@artifact/"
          exclude: ["*.md"]
    - name: api-gateway
      copy:
        - from: "@monorepo/services/gateway/deploy/**"
          to: "@artifact/"
          exclude: ["*.md", "**/charts/**"]
```



Flux Operator

GitOps on Autopilot Mode

fluxoperator.dev

Flux Operator - Overview

The Flux Operator is designed to automate the lifecycle management of Flux components and to streamline the GitOps workflows for Kubernetes clusters.

Highlights

- Declarative cluster bootstrap (FluxInstance CRD)
- Flexible cluster synchronisation (Git repos, OCI registries, S3 buckets)
- High-level abstractions built on top of Flux APIs (ResourceSet CRDs)
- Web UI for deep insights into the delivery pipelines managed by Flux
- AI-assistant GitOps with Flux MCP Server and Agent Skills
- Enterprise Edition with vulnerability remediation & automated patching



Flux Operator

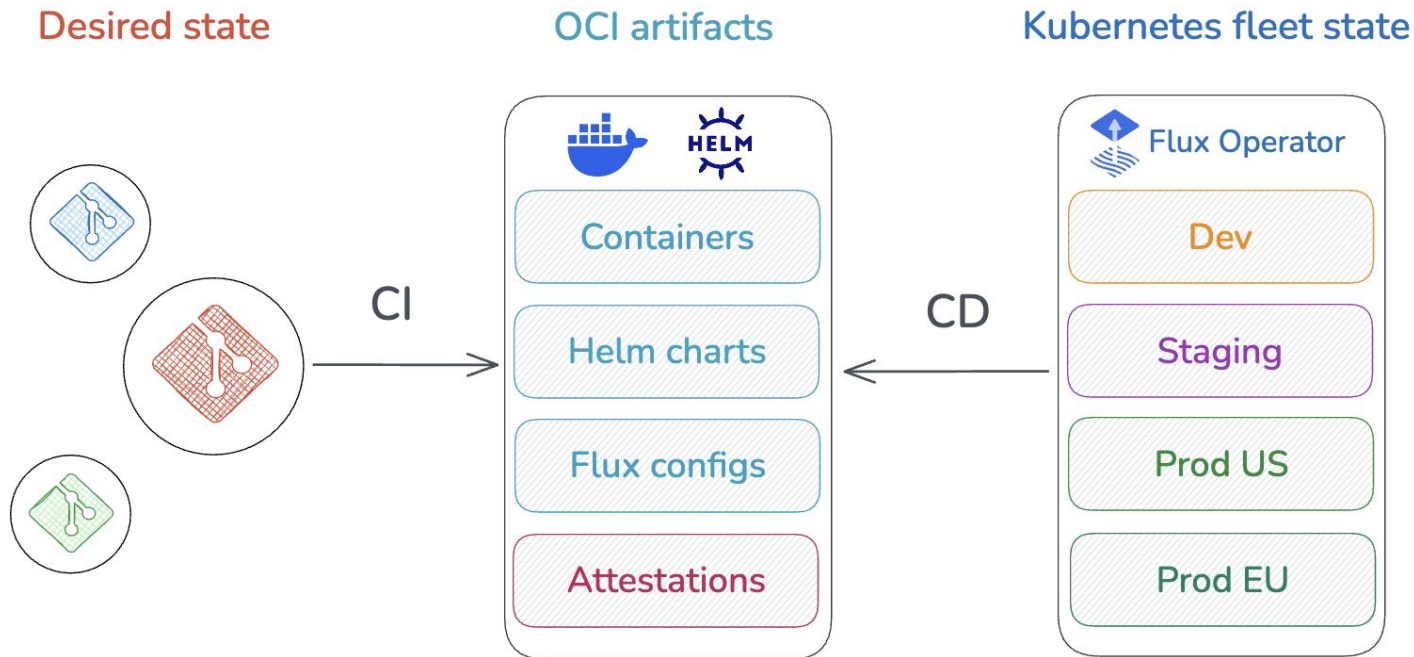
Declarative Cluster Bootstrap

fluxoperator.dev

```
apiVersion: fluxcd.controlplane.io/v1
kind: FluxInstance
metadata:
  name: flux
  namespace: flux-system
spec:
  distribution:
    version: "2.5.x"
    registry: "ghcr.io/fluxcd"
  cluster:
    type: kubernetes
    multitenant: true
    networkPolicy: true
    domain: "cluster.local"
  storage:
    class: "standard"
    size: "10Gi"
  sync:
    kind: OCIRepository
    url: "oci://ghcr.io/my-org/my-fleet-manifests"
    ref: "latest-stable"
    path: "clusters/my-cluster"
    pullSecret: "flux-system"
```

```
apiVersion: fluxcd.controlplane.io/v1
kind: FluxInstance
metadata:
  name: flux
spec:
  components:
    - source-controller
    - kustomize-controller
    - helm-controller
    - notification-controller
    - image-reflector-controller
    - image-automation-controller
  kustomize:
    patches:
      - target:
          kind: Deployment
          name: "(kustomize-controller|helm-controller)"
        patch: |
          - op: add
            path: /spec/template/spec/containers/0/args/-
            value: --concurrent=10
          - op: add
            path: /spec/template/spec/containers/0/args/-
            value: --requeue-dependency=5s
```

Flux Operator - Gitless Bootstrap





Flux Operator

Dynamic Resource Composition

ResourceSet API

The ResourceSet API offers a high-level abstraction for defining and managing Flux resources and related Kubernetes objects as a single unit.

Features

- Reduces the complexity of GitOps repositories
- Multi-instance & multi-cluster provisioning for tenants and workloads
- Advanced dependency management and deploy window scheduling
- Deploy app code and config changes made in Pull Requests to **ephemeral environments** for testing and validation

```
apiVersion: fluxcd.controlplane.io/v1
kind: ResourceSetInputProvider
metadata:
  name: podinfo-release-window
  namespace: apps
spec:
  schedule:
    - cron: "0 9 * * 1-5"
      timeZone: "Europe/Oslo"
      window: "6h"
    - cron: "0 8 * * 1-4"
      timeZone: "America/New_York"
      window: "8h"
  type: ECRArtifactTag
  url: oci://12345.dkr.ecr.eu-west-2.amazonaws.com/podinfo
  filter:
    semver: "≥1.0.0"
    limit: 1
```

```
apiVersion: fluxcd.controlplane.io/v1
kind: ResourceSet
metadata:
  name: podinfo
  namespace: apps
spec:
  inputsFrom:
    - kind: ResourceSetInputProvider
      name: podinfo
  resources:
    - apiVersion: source.toolkit.fluxcd.io/v1
      kind: OCIRepository
      metadata:
        name: podinfo
        namespace: << inputs.provider.namespace >>
      spec:
        interval: 24h
        provider: aws
        url: oci://12345.dkr.ecr.eu-west-2.amazonaws.com/podinfo
        ref:
          digest: << inputs.digest >>
    - apiVersion: kustomize.toolkit.fluxcd.io/v1
      kind: Kustomization
      metadata:
        name: podinfo
        namespace: << inputs.provider.namespace >>
      spec:
        interval: 30m
        prune: true
        wait: true
        timeout: 5m
        sourceRef:
          kind: OCIRepository
          name: podinfo
        path: deploy/
```

```
apiVersion: fluxcd.controlplane.io/v1
kind: ResourceSetInputProvider
metadata:
  name: dev-team-alpha
  namespace: flux-system
  labels:
    fluxcd.io/role: tenant
spec:
  type: Static
  defaultValues:
    tenant: "alpha"
  rbac:
    role: "apps-deployer"
    serviceAccount: "apps-reconciler"
```

```
apiVersion: fluxcd.controlplane.io/v1
kind: ResourceSet
metadata:
  name: tenants
  namespace: flux-system
spec:
  inputsFrom:
    - kind: ResourceSetInputProvider
      selector:
        matchLabels:
          fluxcd.io/role: tenant
  resources:
    - apiVersion: v1
      kind: Namespace
      metadata:
        name: << inputs.tenant >>
    - apiVersion: source.toolkit.fluxcd.io/v1
      kind: OCIRepository
      metadata:
        name: apps
        namespace: << inputs.tenant >>
      spec:
        interval: 24h
        url: oci://registry.internal/<< inputs.tenant >>
    - apiVersion: kustomize.toolkit.fluxcd.io/v1
      kind: Kustomization
      metadata:
        name: apps
        namespace: << inputs.tenant >>
      spec:
        interval: 30m
        prune: true
        sourceRef:
          kind: OCIRepository
          name: apps
        path: ./
```



```
apiVersion: fluxcd.controlplane.io/v1
kind: ResourceSetInputProvider
metadata:
  name: app-pull-requests
  namespace: app-preview
  annotations:
    fluxcd.controlplane.io/reconcileEvery: "1m"
spec:
  type: GitHubPullRequest
  url: https://github.com/org/app
  secretRef:
    name: github-app-auth
  filter:
    labels:
      - "deploy/flux-preview"
  defaultValues:
    chart: "charts/app"
```



```
apiVersion: fluxcd.controlplane.io/v1
kind: ResourceSet
metadata:
  name: app
  namespace: app-preview
spec:
  inputsFrom:
    - apiVersion: fluxcd.controlplane.io/v1
      kind: ResourceSetInputProvider
      name: app-pull-requests
  resources:
    - apiVersion: helm.toolkit.fluxcd.io/v2
      kind: HelmRelease
      metadata:
        name: "app-« inputs.id »"
      spec:
        releaseName: "app-« inputs.id »"
        chart:
          spec:
            chart: "« inputs.chart »"
            reconcileStrategy: Revision
            sourceRef:
              kind: GitRepository
              name: "app-« inputs.id »"
        values:
          image:
            tag: "« inputs.sha »"
          ingress:
            hosts:
              - host: "app-« inputs.id ».example.com"
```

Flux Operator Web UI

Real time monitoring and incident management



All Systems Operational

Cluster in sync with desired state

Last Updated

10:02:37 AM

Cluster Info

Kubernetes v1.33.5-eks-3cfe0ce · 3 nodes

Flux Operator: **v0.37.0-1**

Flux Distribution: **v2.7.5**

Flux CPU Usage

0.01/14.00 cores (0%)

Platform: **linux/amd64**

Controller Pods: **8**

Flux Memory Usage

0.22/8.00 GiB (3%)

Cluster Sync

flux-system

[Kustomization/flux-system/flux-system](#)

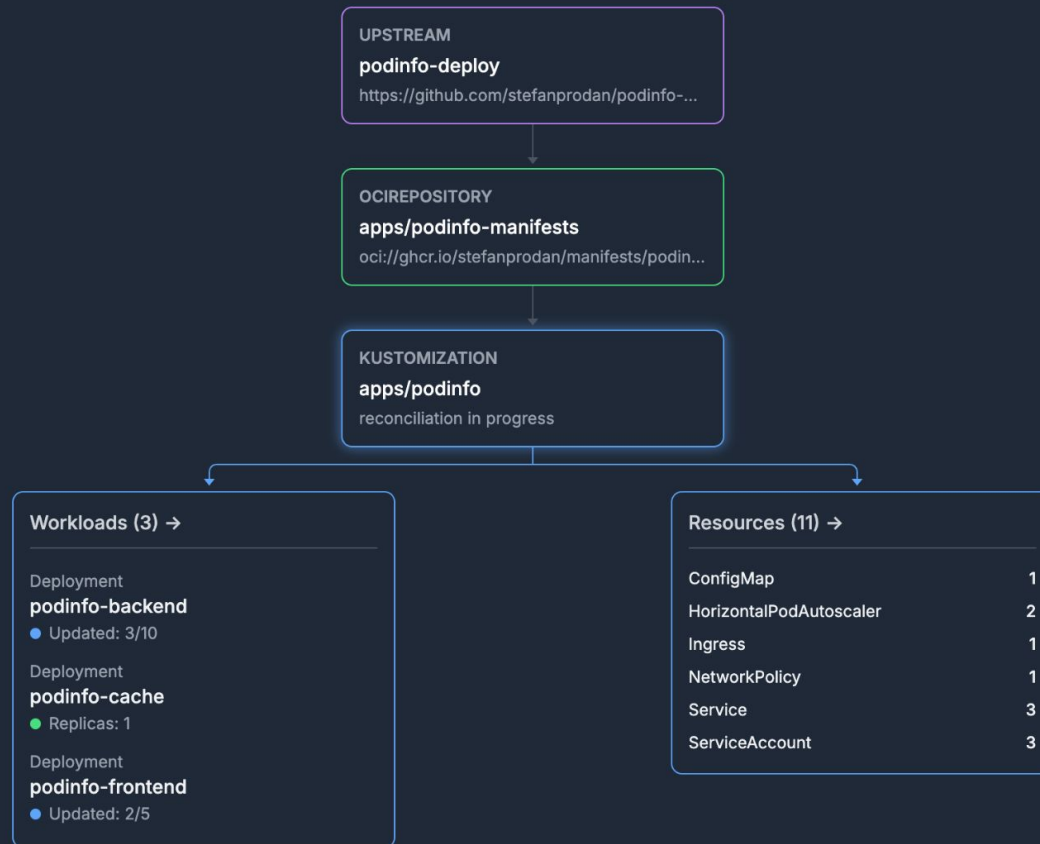
[oci://555034553616.dkr.ecr.eu-west-2.amazonaws.com/github.com/controlplaneio/d2-fleet](#)

[clusters/staging](#)

Applied revision: latest@sha256:d0bd2d018747c92094d54fb2b24fa8a27464f168ca00e8005c380605918b8b6d

Managed Objects

Overview **Graph** Inventory Workloads





Flux MCP Server

Agentic GitOps

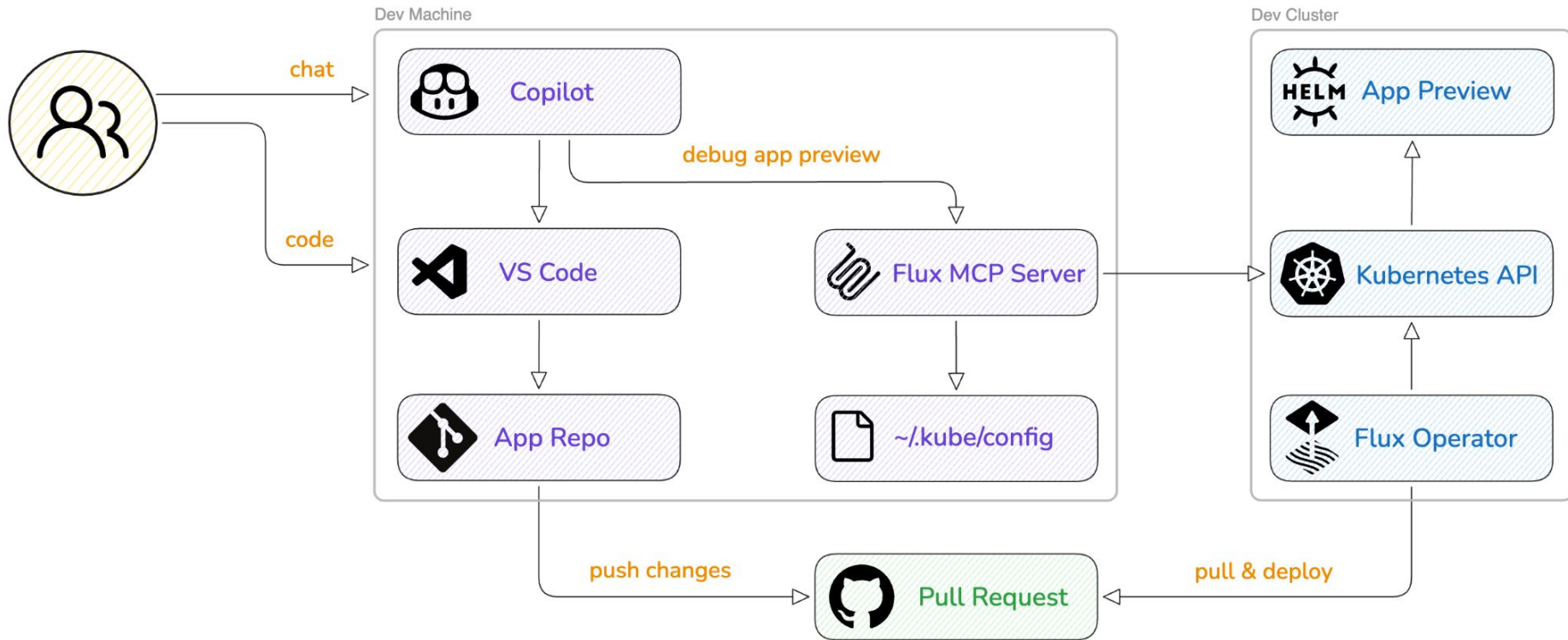
Flux Operator - Model Context Protocol (MCP) Server

The MCP Server connects AI assistants directly to your Kubernetes clusters running Flux Operator, enabling seamless interaction through natural language.

Features

- AI-powered troubleshooting & root cause analysis for issues with apps and infrastructure deployments
- Cross-cluster comparison for Flux configuration and Kubernetes workloads
- GitOps pipeline analysis, generation and refactoring
- Engineers learn Flux concepts in the context of your actual infrastructure

Flux Operator - AI-Assisted Debugging



High Assurance Twist

Gitless GitOps with OCI assumes some ordinary network reachability

- Everything we've covered so far assumes the cluster has egress to the internet
GHCR (or equivalent) pulls, cosign, lookups, ...
- That's a **non-starter** for **defense, nuclear, rail, water, refining**, and in general **OT-based CNIs**
- In **high-assurance OT**, the **security** case **may require hardware-enforced one-way transfer**, not just firewall policy. That is common in defence, nuclear, rail, water, refining, and CNI environments.

So the question: **can we keep the Gitless GitOps wins** on the **other side of an air gap**?

Yes! Bolt on a **data diode** and a **bunch of inspection**!

What breaks, and the target shape

Connected Flux OCI GitOps assumption that a hardware diode breaks:

- Online signature trust breaks at the boundary

Target shape:

CI > external registry > outbound guard > [DIODE] > inbound guard > internal registry > cluster

Low Trust

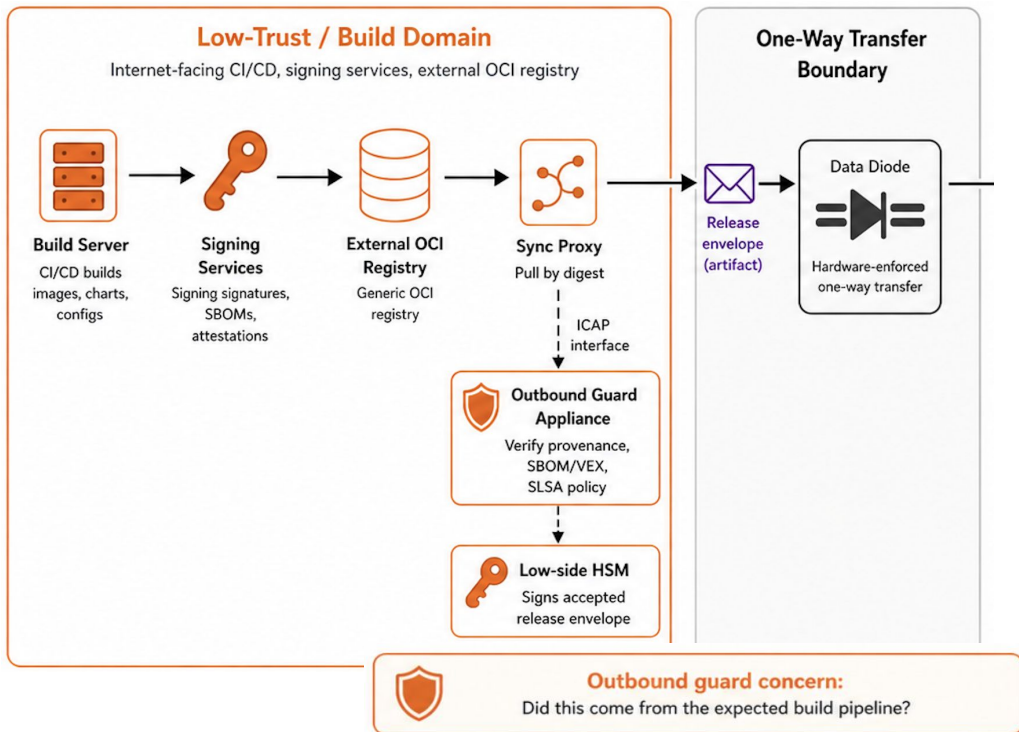
High Trust

Two guards, two concerns:

- **Outbound guard:** did this come from our build pipeline? (**provenance**)
- **Inbound guard:** is this safe to deploy here? (**policy**)

Outbound guard: Provenance

Sync proxy pulls each new bundle (artifact + signature + SBOM + attestations) from the external registry and submits to the **guard** via ICAP (or vendor equivalent)



Ideal policy chain:

- Verify the original CI signature
- Verify expected OIDC identity, repository, workflow, branch, tag, or release ref
- Confirm SBOM is present and signed
- Reject mutable or unresolved external references
- Evaluate CVE policy against SBOM and VEX
- Verify SLSA provenance, where used
- Confirm all referenced images, charts, and dependencies are vendored into the envelope
- **Sign the release envelope digest with HSM**

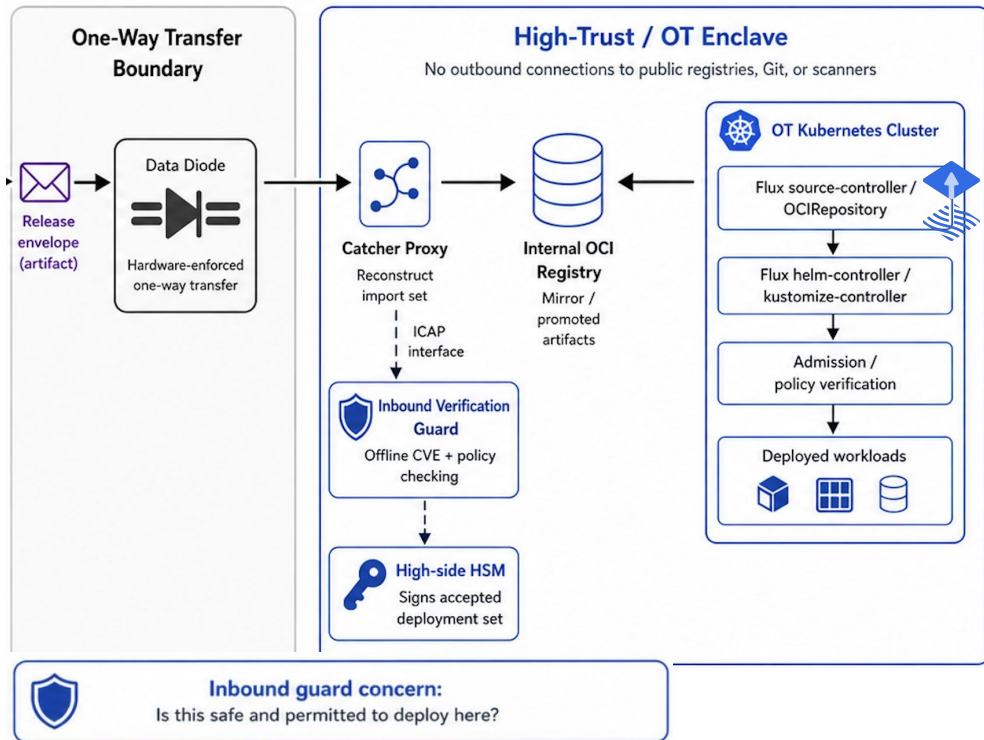
Inbound guard: Import acceptance

Different concern. Provenance $\neq$ deployability.

The inbound guard answers: **can this release envelope be admitted into the high-trust environment?**

Policy chain:

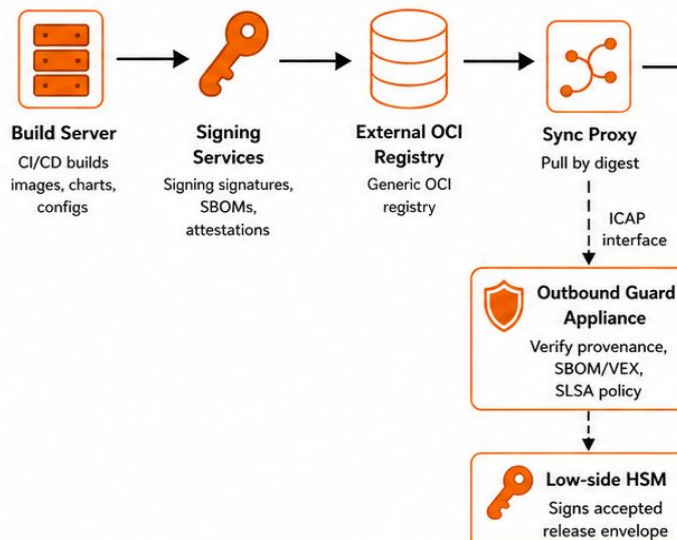
- Verify the low-side HSM signature over the release envelope
- Confirm artifact closure is complete against manifest in signed envelope (images, charts/manifests, signatures, SBOMs, VEX, attestations all present and matching by digest)
- Can scan artifacts using the high-side vulnerability database
- Evaluate SBOM / VEX against high-side policy
- **Sign the accepted release digest with HSM**



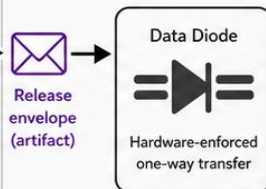
Full Picture

Low-Trust / Build Domain

Internet-facing CI/CD, signing services, external OCI registry

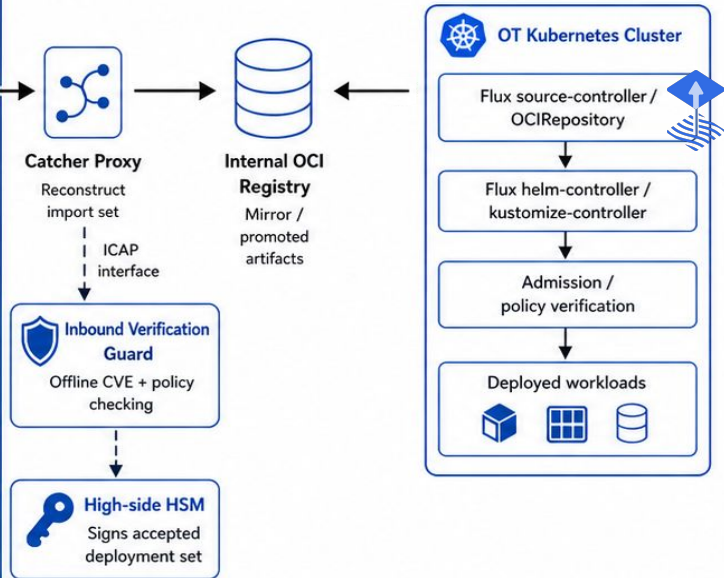


One-Way Transfer Boundary



High-Trust / OT Enclave

No outbound connections to public registries, Git, or scanners



Outbound guard concern:

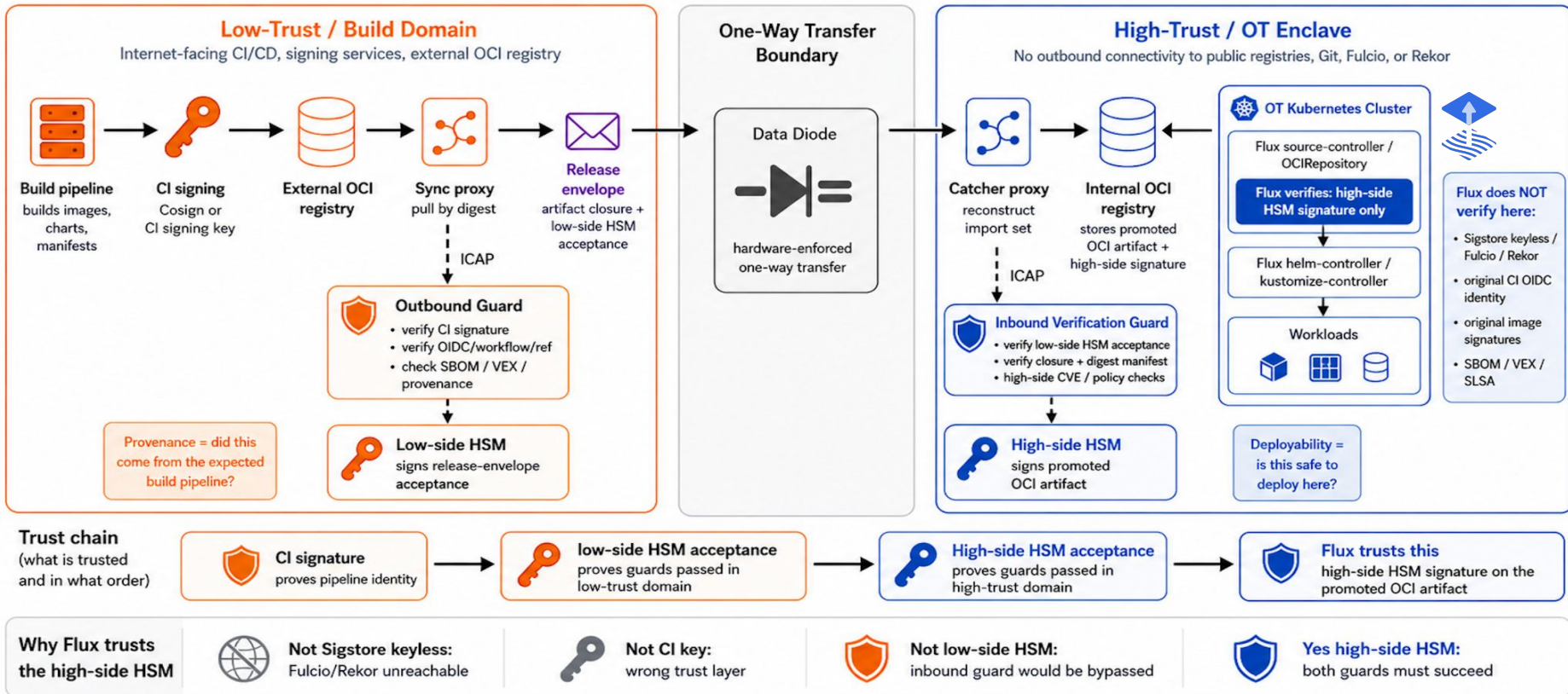
Did this come from the expected build pipeline?



Inbound guard concern:

Is this safe and permitted to deploy here?

Trust Chains

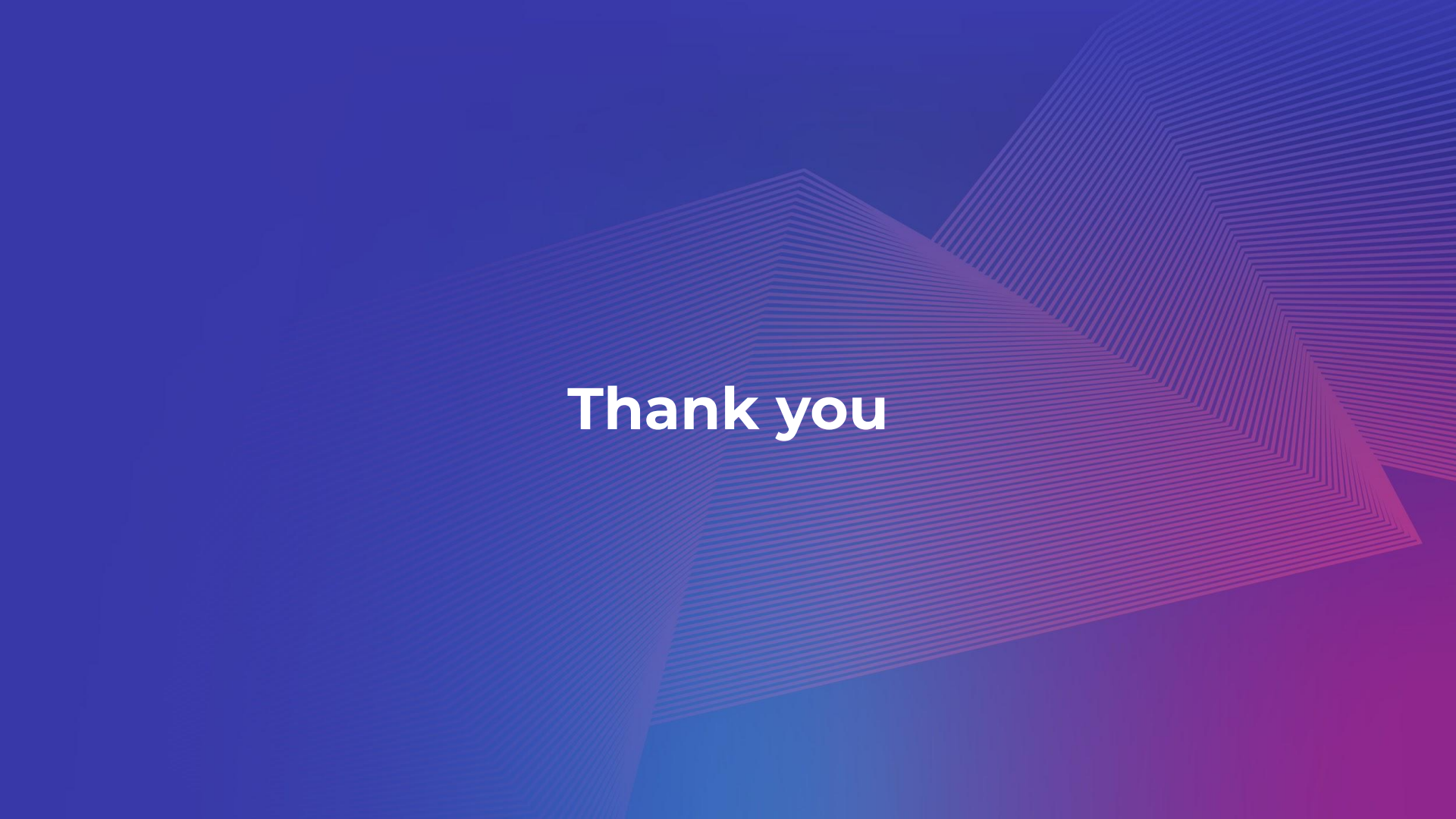


Benefits

- Hardware-enforced one-way transfer
- No outbound dependency from the protected enclave
- Original CI signatures, SBOMs, VEX, and provenance are preserved
- Two guards check different concerns independently
- High-side vulnerability data and asset context never leave the enclave
- Tool-agnostic: works with OCI-aware delivery patterns
- Boundary policy is separated from in-cluster admission policy
- Supports assurance cases for regulated industrial and defense environments

Limitations

- No feedback channel unless a separate reverse diode is added for telemetry-only flows
- Live OCI registry semantics do not cross the diode
- Online Sigstore keyless verification is terminated or converted at the boundary
- Preview environments on the protected production cluster are not a good fit
- Operational surface grows: sync agent, proxies, two guards, HSMs, registry mirror, diode, and admission policy
- Failure handling must be explicit because the high side cannot ask the low side to retry interactively

The background features a solid blue gradient on the left, transitioning into a series of overlapping, semi-transparent geometric shapes on the right. These shapes, which resemble stylized pyramids or chevrons, are composed of numerous thin, parallel lines that create a sense of depth and movement. The colors of these shapes range from a light blue to a vibrant purple.

Thank you